

PRÄDIKATE

Prädikate sind Aussagen über mathematische Objekte, die wahr oder falsch sein können. «Funktionale» Prädikate mit Beispielen Rückgabewerten: $P, Q(n), R(x, y, z)$.

$A \Rightarrow B \Leftrightarrow (\neg A \vee B) \Leftrightarrow (A \wedge B) \vee (\neg A \wedge \neg B) \Leftrightarrow A \vee \neg B$
 $A \Rightarrow B \Leftrightarrow (\neg A \wedge B) \vee (A \wedge \neg B) \vee (A \wedge B) \vee (\neg A \wedge \neg B) \Leftrightarrow A \vee \neg B$

Begriff	Bedeutung
Abtrennungsregel	$(A \wedge (A \Rightarrow B)) \Rightarrow B$
Kommutativität	$(A \wedge B) \Leftrightarrow (B \wedge A)$ $(A \vee B) \Leftrightarrow (B \vee A)$
Assoziativität	$A \wedge (B \wedge C) \Leftrightarrow (A \wedge B) \wedge C$ $A \vee (B \vee C) \Leftrightarrow (A \vee B) \vee C$
Distributivität	$A \wedge (B \vee C) \Leftrightarrow (A \wedge B) \vee (A \wedge C)$ $A \vee (B \wedge C) \Leftrightarrow (A \vee B) \wedge (A \vee C)$
Absorption	$A \vee (A \wedge B) \Leftrightarrow A$ $A \wedge (A \vee B) \Leftrightarrow A$
Idempotenz	$A \vee A = A$ $A \wedge A = A$
Doppelte Negation	$\neg(\neg A) \Leftrightarrow \neg\neg A \Leftrightarrow A$
Konstanten	$W = \text{wahr}$ $F = \text{falsch}$
de Morgan	$\neg(A \wedge B) \Leftrightarrow \neg A \vee \neg B$ $\neg(A \vee B) \Leftrightarrow \neg A \wedge \neg B$

NORMALFORMEN

Normalformen (allgemein, **kanonische Formen**) helfen, das Vergleichsproblem zu lösen (ob zwei Aussagen dieselben sind).

$$P_1 \wedge \dots \wedge P_n = \forall i \in \{1, \dots, n\} (P_i) = \bigwedge_{i=1}^n P_i$$
$$P_1 \vee \dots \vee P_n = \exists i \in \{1, \dots, n\} (P_i) = \bigvee_{i=1}^n P_i$$

NEGATION

Nicht für alle = Es gibt einen Fall, für den nicht
 $\neg \forall i \in \{1, \dots, n\} (P_i) \Leftrightarrow \neg (P_1 \wedge \dots \wedge P_n) \Leftrightarrow \neg P_1 \vee \dots \vee \neg P_n \Leftrightarrow \exists i \in \{1, \dots, n\} (\neg P_i)$

MENGEN

KONSTRUKTION

Grundmenge G , Prädikat $P(x)$. Konstruierte Menge $A = \{x \in G \mid P(x)\}$

OPERATIONEN

Begriff	Bedeutung
Vereinigung	$A \cup B = \{x \mid x \in A \vee x \in B\}$
Schnittmenge	$A \cap B = \{x \mid x \in A \wedge x \in B\}$
Komplement	$\bar{A} = \{x \mid x \notin A\}$
Differenz	$A \setminus B = \{x \in A \mid x \notin B\}$

TUPEL

$$A \times B = \{(a, b) \mid a \in A \wedge b \in B\}$$

n -Tupel:

$$\prod_{i=1}^n A_i = \{(a_1, a_2, \dots, a_n) \mid a_i \in A_i\}$$

BEWEISE

Eine Folge von logischen Schlüssen, die zeigen, dass die Aussage aus den gegebenen Voraussetzungen folgt.

Beweistechnik	Anwendungsfall
Konstruktiver Beweis	Liefert explizite «Lösung» / Algorithmus
Widerspruchsbeweis	Geeignet für Unmöglichkeitss Aussagen. Z.B. $\sqrt{2} \notin \mathbb{Q}$
Vollständige Induktion	Für Aussagen, die für alle $n \in \mathbb{N}$ gelten.

SPRACHEN

Regulär
DEA
NEA
Regex

Kontextfrei
CFG
PDA
Palindrome
{0^n 1^n | n ≥ 0}

Turing-erkennbar
Primzahlen
{a^n b^n c^n | n ≥ 0}

Begriff	Bedeutung
Alphabet	Eine nichtleere endliche Menge Σ heisst Alphabet . Die Elemente von Σ heissen Zeichen .
Wort	Eine Zeichenkette der Länge n ist ein n -Tupel in $\Sigma^n = \Sigma \times \dots \times \Sigma$. Ein Element von Σ^n heisst Wort der Länge n . Wörter haben immer endliche Länge.
Leeres Wort	Die Zeichenkette $\epsilon \in \Sigma^0 = \{\epsilon\}$ der Länge 0 heisst das leere Wort .
Menge aller Wörter	Leeres Wort ist immer drin $\Sigma^* = \{\epsilon\} \cup \Sigma \cup \Sigma^2 \cup \Sigma^3 \cup \dots = \bigcup_{k=0}^{\infty} \Sigma^k$
Abgekürzte Schreibweisen von Wörtern	$\langle A, u, t, o, S, p, r \rangle = \text{AutoSpr}$ $a^3 b^5 = \text{aaabbbbbb}$ $b^5 a^3 = \text{bbbbbaaa}$
Sprache	Teilmenge $L \subset \Sigma^*$

WORTLÄNGE	
Begriff	Bedeutung
Länge des Wortes	$w \in \Sigma^n \Rightarrow w = n$, n ist Länge des Wortes w
Anzahl Zeichen	Sei $w \in \Sigma^n, a \in \Sigma$. Dann ist $ w _a$ die Anzahl Zeichen a im Wort w

Beispiel 1: Wortlänge

$ \epsilon = 0$	$ 01010 _1 = 2$	$ a^3 b^5 = 8$
$ 01010 _0 = 3$	$ 01010 _3 = 0$	$ (1291)^7 = 7 \cdot 1291 = 7 \cdot 4 = 28$

Beispiel 2: Sprache

$L = \Sigma^* \subset \Sigma^*$
 $L = \emptyset \subset \Sigma^*$, die leere Sprache
 $\Sigma = \{0, 1\}$, Sprache aller Binärstrings: $L = \Sigma^*$

$\Sigma = \text{Unicode}, J = \{w \in \Sigma^* \mid w \text{ wird vom Java-Compiler akzeptiert}\}$
 $M = \text{eine "Maschine"}, L(M) = \{w \in \Sigma^* \mid w \text{ wird von } M \text{ akzeptiert}\}$

Beobachtungen 1

1.1. $|w^n| = n \cdot |w|$
1.2. Zu jeder Maschine M gibt es eine Sprache $L(M)$

DETERMINISTISCHE ENDLICHE AUTOMATEN (DEA)

Definition	Tabellenform	Zustandsdiagramm von A
------------	--------------	------------------------

$A = (Q, \Sigma, \delta, q_0, F)$

- Zustände: $Q = \{q_0, q_1, \dots, q_n\}$
- Übergangsfunktion: $\delta: Q \times \Sigma \rightarrow Q$
- Startzustand: $q_0 \in Q$
- Akzeptierzustände: $F \subset Q$

Wichtig: Ein Pfeil für jedes Zeichen in jedem Zustand für validen DEA

WÖRTER EINER REGULÄREN SPRACHE

ÜBERGANGSFUNKTIONEN FÜR WÖRTER

$\delta: Q \times \Sigma^* \rightarrow Q: (q, w = a_1, \dots, a_n) \mapsto \delta(\dots \delta(\delta(q, a_1), a_2), \dots, a_n)$

Übergänge ausgehend von q für Zeichen $a_1, \dots, a_n$ nacheinander anwenden.

WORT AKZEPTIEREN

Der DEA $A = (\Sigma, Q, q_0, \delta, F)$ **akzeptiert** das Wort $w \in \Sigma^*$, wenn er A von Startzustand in einen Akzeptierzustand $\delta(q_0, w) \in F$ überführt.

AKZEPTIERTE SPRACHE, REGULÄRE SPRACHE

Gegeben ein DEA $A = (\Sigma, Q, q_0, \delta, F)$. Die von A **akzeptierte Sprache** ist

$$L(A) = \{w \in \Sigma^* \mid A \text{ akzeptiert } w\} = \{w \in \Sigma^* \mid \delta(q_0, w) \in F\}$$

Die Sprache $L \subset \Sigma^*$ heisst **regulär**, wenn es einen DEA A gibt mit $L(A) = L$

Beispiel 3: DEA, der die Sprache $L = \{w \in \Sigma^* \mid w \text{ ist eine ungerade Zahl im Dreiersystem}\}$ akzeptiert

Oder: genügend lange Wörter ($|w| \geq N$) einer regulären Sprache ($w \in L$) können alle in einem Anfangsstück der Länge N ($|xy| \leq N$) aufgepumpt werden ($xy^k z \in L$).

Was zeichnet reguläre Sprachen aus?

- Reguläre Ausdrücke
- Lange Wörter: * kommt im regulären Ausdruck vor
- Blöcke mit * können beliebig oft wiederholt werden
- => Wörter können «aufgepumpt» werden

MYHILL-NERODE AUTOMAT

Für ein Wort $w \in \Sigma^*$ setze $L(w) = \{w' \in \Sigma^* \mid ww' \in L\}$. Insbesondere $L(\epsilon) = L$

Gegeben: reguläre Sprache L über Σ . Rekonstruiere A mit:

- $Q = \{L(w) \mid w \in \Sigma^*\}$
- $q_0 = L(\epsilon) = L$
- $F = \{L(w) \in Q \mid \epsilon \in L(w)\}$
- $\delta(L(w), a) = L(wa)$

Gleicher Zustand $\Leftrightarrow$ gleiches $L(w)$

Beispiel 4: $\Sigma = \{0, 1\}, L = \{w \in \Sigma^* \mid |w|_0 \text{ gerade}\}$

w	$L(w)$	Q
ϵ	$L(\epsilon) = L$	q_0
0	$L(0) = \{w \in \Sigma^* \mid w _0 \text{ ungerade}\}$	q_1
1	$L(1) = \{w \in \Sigma^* \mid w _0 \text{ gerade}\}$	q_0

MINIMALAUTOMAT

Ziel: Nicht unterscheidbare Zustände zusammenlegen

Vorgehen:

- Akzeptierzustand $\neq$ Nichtakzeptierzustand
- Iteration: alle unterscheidbaren Paare suchen
- Verbleibende Paare sind ununterscheidbar $\rightarrow$ zusammenlegen

Beispiel 5

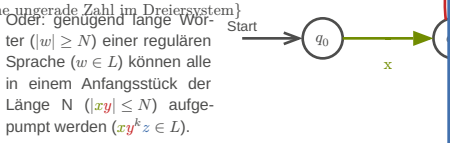
Algorithmus

- $q \in F, q \notin F$
- Unterscheidbar nach Übergang
- Iteration bis keine Änderung mehr
 $\Rightarrow q_1$ und q_2 sind nicht unterscheidbar

PUMPING LEMMA

Ist L eine reguläre Sprache, dann gibt es $N \in \mathbb{N}$, die pumping length so, dass jedes Wort $w \in L$ mit $|w| \geq N$ in drei Teile $w = xyz$ zerlegt werden kann mit:

- $|xy| \leq N$
- $|y| > 0$
- $xy^k z \in L \forall k \in \mathbb{N}$



Was zeichnet reguläre Sprachen aus?

- Reguläre Ausdrücke
- Lange Wörter: * kommt im regulären Ausdruck vor
- Blöcke mit * können beliebig oft wiederholt werden
- => Wörter können «aufgepumpt» werden

BEWEIS

Behauptung: Die Sprache $L \subset \Sigma^*$ ist nicht regulär.

Beweis (Widerspruch):

- Annahme: L ist regulär
- Gemäss Pumping Lemma gibt es die Pumping Length N
 - Es darf keine Annahme über die konkrete Grösse von N gemacht werden!
- Wähle ein Wort $w \in L$ mit $|w| \geq N$
 - Die Definition **muss** N verwenden!
- Aufteilung des Wortes gemäss Pumping Lemma
 - $w = xyz, |xy| \leq N, |y| > 0$
- Auswirkung des Pumpens
 - $xy^k z \notin L$ für mindestens ein $k \in \mathbb{N}$ (mit Begründung!)
- Widerspruch und Schlussfolgerung, dass die Annahme nicht zutreffen kann

Beispiel 6: $L = \{0^n 1^n \mid n \geq 0\}$ ist nicht regulär

- Annahme: L ist regulär
- $\exists N \in \mathbb{N}$, Pumping Length
- $w = 0^N 1^N$
- Unterteilung $w = xyz$

TODO: reduce confusion

Diagram showing string representations with boxes for x, y, z and their lengths.

5) Pumpen: nur die Anzahl der 1 wird erhöht, Anzahl 0 bleibt
6) $xy^k z \notin L$ für $k \neq 1$, im Widerspruch zum Pumping Lemma

NICHTDETERMINISTISCHE ENDLICHE AUTOMATEN (NEA)

Definition

$A = (Q, \Sigma, \delta, q_0, F)$

- Endliche Menge von Zuständen: Q
- Alphabet: Σ
- Übergangsfunktion: $\delta: Q \times \Sigma \rightarrow P(Q)$
- Akzeptierzustände: $F \subset Q$

Akzeptieren

Ein NEA A **akzeptiert** das Wort $w \in \Sigma^*$, wenn es eine **Wahl** von Übergängen gibt, derart, dass das Wort w den Automaten in einen Akzeptierzustand überführt.

Faustregeln

- Nur genau diejenigen Pfeile einzeichnen, die man zum Akzeptieren braucht.
- Erlaube Alternativen

Beispiel 7: $L = \{w \in \Sigma^* \mid w \text{ endet mit zwei 0}\}$

DEA-Lösung

NEA-Lösung

Diagram showing two automata for the language L. The NEA solution has a non-deterministic transition from q1 to q0.

UMGANG MIT MEHRDEUTIGEN ÜBERGÄNGEN

a-Übergang von q_1 aus nicht eindeutig

Welchen Übergang soll man nehmen?

- Beide Möglichkeiten probieren und akzeptieren, falls eine der Möglichkeiten auf einen Akzeptierzustand führt.
- Zufallsgenerator (probabilistischer endlicher Automat, PEA)



4) Startvariable: S

Es ist nicht garantiert, dass es für die Ableitung eines Wortes nur einen einzigen Syntaxbaum gibt. Man sagt, die Grammatik ist **nicht eindeutig**, wenn sie mehrere Syntaxbäume für die gleichen Wörter erlaubt.

REGULÄRE OPERATIONEN
Die Klasse der kontextfreien Sprachen ist abgeschlossen unter regulären Operationen.

Reguläre Sprachen sind kontextfrei.

Grammatik für reguläre Operationen

Seien L_1 und L_2 kontextfreie Sprachen mit Grammatiken $G_i = (V_i, \Sigma, R_i, S_i)$. Wir nehmen an, dass die Variablen- und Regelmengen disjunkt sind, also $V_1 \cap V_2 = \emptyset \wedge R_1 \cap R_2 = \emptyset$. Zudem sei S_0 eine Variable, die nicht in $V_1 \cup V_2$ enthalten ist. Dann gilt:

Alternative $L_1 \mid L_2$ ist kontextfrei mit der Grammatik

$G = (V_1 \cup V_2 \cup \{S_0\}, \Sigma, R = R_1 \cup R_2 \cup \{S_0 \rightarrow S_1 \mid S_2\}, S_0)$

Verkettung $L_1 L_2$ ist kontextfrei mit der Grammatik

$G = (V_1 \cup V_2 \cup \{S_0\}, \Sigma, R = R_1 \cup R_2 \cup \{S_0 \rightarrow S_1 S_2\}, S_0)$

***Operation L_1^* ist kontextfrei mit der Grammatik**

$G = (V_1 \cup \{S_0\}, \Sigma, R = R_1 \cup \{S_0 \rightarrow S_0 S_1, S_0 \rightarrow \varepsilon\}, S_0)$

KONTEXTFREI
Regeln ohne Kontext

Es spielt keine Rolle, in welchem Kontext das Zeichen A vorkommt, die Regel kann immer angewendet werden

$S \rightarrow A \mid C$
 $A \rightarrow a$
 $A \rightarrow aAb$
 $C \rightarrow bA$

Regeln mit Kontext

Je nach **Kontext** kann eine Regel nicht unbedingt angewendet werden

$S \rightarrow C \mid aC \mid bC$
 $aC \rightarrow A$
 $bC \rightarrow B$
 $S \rightarrow aC \rightarrow A$
 $S \rightarrow bC \rightarrow B$
 $S \rightarrow C \rightarrow ?$

CHOMSKY-NORMALFORM (CNF)

Eine CFG ist in **Chomsky-Normalform**, wenn S auf der rechten Seite nicht vorkommt und jede Regel von der Form $A \rightarrow BC$ oder $A \rightarrow a$ ist, zusätzlich ist die Regel $S \rightarrow \varepsilon$ erlaubt.

Umwandlung

- Neue Startvariable $S_0 \rightarrow S$ (wenn nötig)
- ε -Regeln: $A \xrightarrow{\varepsilon} AC \Rightarrow A$ kann weggelassen werden $\Rightarrow \begin{cases} B \rightarrow AC \\ \rightarrow \cdot C \end{cases} \xrightarrow{A \rightarrow B}$
- Unit-Rules: $B \rightarrow CP \Rightarrow$ aus A kann man wie aus B auch CD machen $\Rightarrow \begin{cases} A \rightarrow CD \\ \rightarrow \cdot CD \end{cases}$
- Verkettungen: $A \rightarrow u_1 u_2 \dots u_n$ ersetzen durch $A \rightarrow u_1 A_1, A_1 \rightarrow u_2 A_2, \dots, A_{n-2} \rightarrow u_{n-1} u_n$ und falls u_i ein Terminalsymbol ist: $A_{i-1} \rightarrow U_i A_i, U_i \rightarrow u_i$.

Folgerungen

- Ableitung eines Wortes $w \in L(G)$ ist immer in $2|w| - 1$ Regelanwendungen möglich. Beweis:
 - $|w| - 1$ Regeln der Form $A \rightarrow BC$ um aus S ein Wort aus $|w|$ Variablen zu erzeugen
 - $|w|$ Regeln der Form $A \rightarrow a$ um das Wort w zu erzeugen $\Rightarrow$ insgesamt $2|w| - 1$ Regelanwendungen
- Deterministischer Parse-Algorithmus mit Laufzeit $O(|w|^3) \rightarrow$

Beispiel 9:
 $S \rightarrow ASA \mid aB$
 $A \rightarrow B \mid S$
 $B \rightarrow b \mid \varepsilon$

1) Startvariable
 $S_0 \rightarrow S$
 $S \rightarrow ASA \mid aB$
 $A \rightarrow B \mid S$
 $B \rightarrow b \mid \varepsilon$

2) ε -Regel
 $S_0 \rightarrow S$
 $S \rightarrow ASA \mid aB \mid a$
 $A \rightarrow B \mid \varepsilon \mid S$
 $B \rightarrow b$

3) Unit-Regel
 $S_0 \rightarrow ASA \mid SA \mid AS \mid aB \mid aS_0 \rightarrow ASA \mid SA \mid AS \mid aB \mid a$
 $S \rightarrow ASA \mid SA \mid AS \mid aB \mid a$
 $A \rightarrow B \mid ASA \mid SA \mid AS \mid aB \mid a$
 $B \rightarrow b$

4) Komplexe Regeln
 $S_0 \rightarrow AA_1 \mid SA \mid AS \mid UB \mid a$
 $S \rightarrow AA_1 \mid SA \mid AS \mid UB \mid a$
 $A \rightarrow b \mid AA_1 \mid SA \mid AS \mid UB \mid a$
 $AA_1 \rightarrow SA$
 $U \rightarrow a$
 $B \rightarrow b$

PARSING
COCKE-YOUNGER-KASAMI ALGORITHMUS (CYK)
Gegeben
1) Grammatik $G = (V, \Sigma, R, S)$
2) Variable $A \in V$
3) Wort $w \in \Sigma^*$

Frage

Ist w ableitbar? Formell geschrieben: $A \xRightarrow{*} w$
– Spezialfall $w = \varepsilon$: $A \xRightarrow{*} \varepsilon \Leftrightarrow A \rightarrow \varepsilon \in R$ (Epsilonregel, $A = S$)
– Spezialfall $|w| = 1$: $A \xRightarrow{*} w \Leftrightarrow A \rightarrow w \in R$ (Terminalsymbolregel)
– Fall $|w| > 1$:

$A \xRightarrow{*} w \Rightarrow \exists \begin{cases} A \rightarrow BC \in R \\ w = w_1 w_2, w_i \in \Sigma^* \end{cases} \text{ mit } \begin{cases} B \xRightarrow{*} w_1 \\ C \xRightarrow{*} w_2 \end{cases}$

ABLEITUNGSDREIECK
Ideen

- Parse Tree aus $A \rightarrow BC$ und $T \rightarrow t$
- Variablen in Tabelle füllen

Prinzip
– Einem Teilwort entspricht ein Feld der Tabelle (rot hinterlegt)
– Das Feld wird mit den Variablen gefüllt, aus denen das Teilwort abgeleitet werden kann.

Beispiel
Parzen des Wortes $()()()$

Definitionen
Die Felder (k, l_1) und $(k + l_1, l - l_1)$ heißen **korrespondierende Felder** im Ab-

leitungs-dreieck des Feldes (k, l) .

TODO:
rekursion vs iteration (buch 149)

BACKUS-NAUR-FORM (BNF)
Spezifikation der Regeln in maschinen-lesbarer Form:

- Variablen: **<variablen-name>**
- Einzelne Zeichen: A
- Zeichenketten: **'BEISPIEL'**
- Regeln: **<variablen-name> ::= Ausdruck**
- Ausdrücke sind Folgen von Variablen, einzelnen Zeichen oder Zeichenketten, getrennt durch $|$

Beispiel 10: BNF für Expression-Term-Factor Grammatik

Ausgangsgrammatik

expression $\rightarrow$ expression + term | term
term $\rightarrow$ term * factor | factor
factor $\rightarrow$ (expression) | N
 $N \rightarrow NZ \mid Z$
 $Z \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

Backus-Naur-Form

<expression> ::= <expression> + <term> | <term>
<term> ::= <term> * <factor> | <factor>
<factor> ::= (<expression>) | <number>
<number> ::= <number> <digit> | <digit>
<digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

EXTENDED BACKUS-NAUR-FORM (EBNF)

Definition

- Literale in Anführungszeichen oder Apostroph
- = statt ::=
- Variablen ohne < und >, dürfen Leerzeichen enthalten
- Komma für Verkettung
- Regel-Endzeichen
- Kommentare (* ... *)
- Optionale Wiederholung { ... }
- Option [...]
- Gruppierung (...)
- Ausnahme: -

Achtung!

- Keine Operatorpriorisierung
- Zweideutigkeit!
- Keine eindeutige Evaluation!

Beispiel 11: EBNF für Expression-Term-Factor Grammatik

expression = expression, "+", term | term
term = term, "*", factor | factor
factor = "(", expression, ")" | number
number = digit, { digit }
digit = "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

RECHTSREKURSION
Expression-Term-Factor-Grammatik verwendet Links-Rekursion $\Rightarrow$ korrekte Auswertung von links nach rechts. Parsetree wertet aber Ausdrücke von rechts nach links aus! Alternative Expression-Term-Factor definition mit Rechtsrekursion statt Linksrekursion:

expression $\rightarrow$ term expression'
expression' $\rightarrow$ + term expression' | ε
term $\rightarrow$ factor term'
term' $\rightarrow$ * factor term' | ε
factor $\rightarrow$ (expression) | N
 $N \rightarrow NZ \mid Z$
 $Z \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

TODO:
grid $\rightarrow$ table

STACK
Unendlich grosser Speicher

- Immer nur das oberste Element sichtbar
- Beliebig tief

STACKAUTOMAT / PUSHDOWN AUTOMATON (PDA)
Kann Sprachen akzeptieren, die nicht regulär sind, ist aber nicht deterministisch. **Beachte:** $\Gamma \neq \Sigma$ möglich

Definition
Stackautomat $(Q, \Sigma, \Gamma, \delta, q_0, F)$

Übergänge

$P =$

q vom Input
 b vom Stack entfernen (Bedingung)
 c auf den Stack

1) Q : Zustände
2) Σ : Eingabe-Alphabet
3) Γ : Stack-Alphabet
4) $\delta: Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \rightarrow P(Q \times \Gamma_\varepsilon)$
5) $q_0 \in Q$: Startzustand
6) $F \subset Q$: Akzeptierzustände

Beispiel 12: Stackautomat für die Sprache $\{0^n 1^n \mid n \geq 0\}$

Start q_0 $\xrightarrow{\varepsilon, \varepsilon \rightarrow \$}$ q_1 $\xrightarrow{\varepsilon, \varepsilon \rightarrow \varepsilon}$ q_2 $\xrightarrow{\varepsilon, \$ \rightarrow \varepsilon}$ q_3

TODO:
Book 167-168, shift/reduce

TODO:
diagrams? (slides 8/12)

ABLEITUNG AUS CNF
Ist L eine kontextfreie Sprache, dann gibt es einen Stackautomaten P , der L akzeptiert, $L = L(P)$.

Grundgerüst

Regel $A \rightarrow BC$ $\varepsilon, A \rightarrow C$ $\varepsilon, A \rightarrow B$
Regel $A \rightarrow a$ $\varepsilon, a \rightarrow R$
Regel $S \rightarrow \varepsilon$ $\varepsilon, S \rightarrow \varepsilon R$
 $\forall a \in \Sigma$ $a, a \rightarrow \varepsilon$

PDA $\rightarrow$ CFG
Variablen
Wörter beschreiben Pfade durch den Automaten: Variable A_{pq} = Wörter, die von p nach q führen mit leerem Stack

Regeln
Regeln beschreiben, wie sich Wege zerlegen lassen: $A_{pq} \rightarrow A_{pr} A_{rq}$ = Wege von p nach q können verstanden werden als Wege von p nach r und von dort nach q

Stackautomat standardisieren

TODO:
better diagrams, book 181

- Nur ein Akzeptierzustand: Neuen Akzeptierzustand q_a und Übergänge von allen vorherigen Endzuständen zu q_a erstellen.
 $\forall q \in F$: Start $\rightarrow q \rightarrow q_a$
- Stack leeren: Mit einem Element $\$$ erweitern, sodass garantiert werden kann, dass der Stack am schluss leer ist.
 $\varepsilon, \cdot \rightarrow \varepsilon$
 $\varepsilon, \varepsilon \rightarrow \$$
 $\varepsilon, \$ \rightarrow \varepsilon$

3) Jeder Übergang legt entweder ein Zeichen auf den Stack oder entfernt eines

$a, b \rightarrow c$
 $a, b \rightarrow \varepsilon$
 $a, \varepsilon \rightarrow \varepsilon$
 $a, \varepsilon \rightarrow t$

TODO:
Book 182,183

Beispiel 13

TODO:
visualization

Grammatik ablesen
Ausgangspunkt: standardisierter PDA mit Startzustand q_0 und $F = \{q_a\}$.

- Startvariable: $A_{q_0 q_a}$
- Regeln:

Beispiel 14

TODO:

NICHT KONTEXTFREIE SPRACHEN
PUMPING LEMMA
Pumping Lemma für CFL
Ist L eine CFL, dann gibt es eine Zahl N , die die Pumping Length, derart, dass jedes Wort $w \in L$ mit $|w| \geq N$ zerlegt werden kann in fünf Teile $w = uvxyz$ derart, dass

- $|vxy| > 0$
- $|vxy| \leq N$
- $uv^k xy^k z \in L \forall k \in \mathbb{N}$

REDUKTION

Reduktionsabbildung

Berechenbare Abbildung $f : \Sigma^* \rightarrow \Sigma^*$ so, dass

$w \in A \Leftrightarrow f(w) \in B$

Notation: $f : A \leq B$ heisst «A leichter entscheidbar als B»

Entscheidbarkeit

Falls B entscheidbar, dann $f : A \leq B \Rightarrow A$ entscheidbar

Beweis

Ist H ein Entscheider für B , dann ist $H \circ f$ ein Entscheider für A .

Folgerung

A nicht entscheidbar, $A \leq B \Rightarrow B$ nicht entscheidbar

VERGLEICH DER ENTSCHEIDBARKEIT VON SPRACHEN

Beispiel 18

TODO:

SATZ VON RICE
LÖSUNGSMETHODEN FÜR ENTSCHEIDUNGSPROBLEME

TODO:

book 273..

